# A Perceptual-Dynamic Gap Bridging Pipeline for Quadrupedal Visual Navigation using 3D Gaussian Splatting and RL-Induced Jitter

Minseok Lee[1], Seongyu Han[1], Heejae Moon[1], and Sung Soo Hwang[1*]

[1]School of Artificial Intelligence, Computer and Electrical Engineering,
Handong Global University, Pohang 37554, Republic of Korea
({glen, tjsrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * Corresponding author

**Abstract:** Directly validating and deploying autonomous visual navigation systems on physical quadrupedal robots in real-world settings presents severe challenges, specifically the perception-level sim-to-real gap (lack of visual photorealism in synthetic environments) and locomotion-induced camera shaking (ego-motion jitter). This vibration degrades RGB-D feature tracking and depth consistency. To resolve these challenges safely and efficiently, this paper proposes a practical pipeline that bridges these perceptual and dynamic gaps to enable virtual validation and direct parameter optimization. In the virtual sandbox stage, we reconstruct a high-fidelity 3D Gaussian Splatting (3DGS) digital twin of an actual campus corridor and utilize a reinforcement learning (RL) locomotion policy in simulation to actively emulate walking dynamics and generate representative camera bobbing noise. Under these synchronized visual and dynamic disturbances, the parameters of the RTAB-Map visual SLAM and Nav2 stacks are pre-tuned and optimized. In the deployment stage, the RL policy is bypassed, and navigation velocity commands are routed directly to the physical robot's built-in Sport API via a custom ROS 2 bridge. Experimental results in the actual corridor demonstrate that parameters optimized under the simulated RL-induced jitter transfer directly to the physical platform, achieving stable visual mapping and autonomous obstacle avoidance without real-world parameter recalibration.

**Keywords:** Quadrupedal Locomotion, Visual SLAM, Sim-to-Real Pipeline, 3D Gaussian Splatting, Ego-motion Jitter, Virtual Sandbox.

## 1. INTRODUCTION

Autonomous quadrupedal robots have attracted substantial attention due to their superior mobility in traversing challenging and unstructured terrains compared to conventional wheeled systems. Achieving fully autonomous operations in indoor environments requires the seamless integration of stable legged locomotion and high-fidelity simultaneous localization and mapping (SLAM). Conventionally, indoor navigation has relied on Light Detection and Ranging (LiDAR) sensors. However, LiDARs are cost-prohibitive, heavy, and power-intensive, which makes visual navigation using a single, lightweight RGB-D camera a highly desirable, cost-effective alternative.

Nevertheless, deploying visual navigation using a single RGB-D camera on a legged platform introduces two critical sim-to-real challenges. First is the *perceptual sim-to-real gap*: standard physics simulators generate synthetic environments that lack photo-realistic visual textures and depth fidelity, causing feature-based visual SLAM to fail in the real world due to severe matching errors. Second is *locomotion-induced ego-motion noise* (commonly referred to as bobbing noise): legged locomotion inherently generates periodic, high-frequency camera oscillations that cause motion blur and introduce temporal depth discrepancies (phantom obstacles). Because quadrupedal robots are highly expensive, sophisticated platforms, directly tuning navigation parameters on physical hardware to mitigate these disturbances is heavily constrained. Real-world experiments face strict time limits due to battery capacity, and any control or mapping failure during parameter tuning poses severe risks of collision-induced physical damage to the costly robot.

To address these limitations, we propose a practical, bifurcated gap-bridging pipeline for quadrupedal visual navigation utilizing a single front-facing RGB-D camera (Intel RealSense D435i). Our framework decouples virtual-space parameter verification from physical-space system deployment. Crucially, by using the same physical D435i sensor for both offline 3DGS scene reconstruction and online visual SLAM, we eliminate the cross-device perceptual gap. In the simulation phase, we reconstruct a photorealistic digital twin of an indoor corridor using 3DGS. Because the robot's proprietary, low-level controller cannot be integrated directly into the physics simulator, we train a model-free RL locomotion policy in Isaac Lab to emulate physical gait characteristics, thereby generating representative camera ego-motion noise in the virtual environment. Under this synthesized visual and dynamic sandbox, we optimize the parameters of the RTAB-Map and Nav2 frameworks. Upon physical deployment, the deep neural network policy is bypassed, and navigation commands are routed to the manufacturer's built-in Sport API via a lightweight ROS 2 bridge, reserving the edge computer's resource budget for visual SLAM and navigation.

The contributions of this work are summarized as follows:

1. We present a system integration pipeline that bridges the perceptual sim-to-real gap by incorporating 3DGS and *fVDB* reconstruction into a physics simulator, establishing a photorealistic visual validation sandbox that eliminates cross-device sensor discrepancies by using a single homogeneous camera.
2. We bridge the dynamic sim-to-real gap by training an RL locomotion policy to serve as an active camera jitter generator, recreating quadruped-specific walking oscilla-

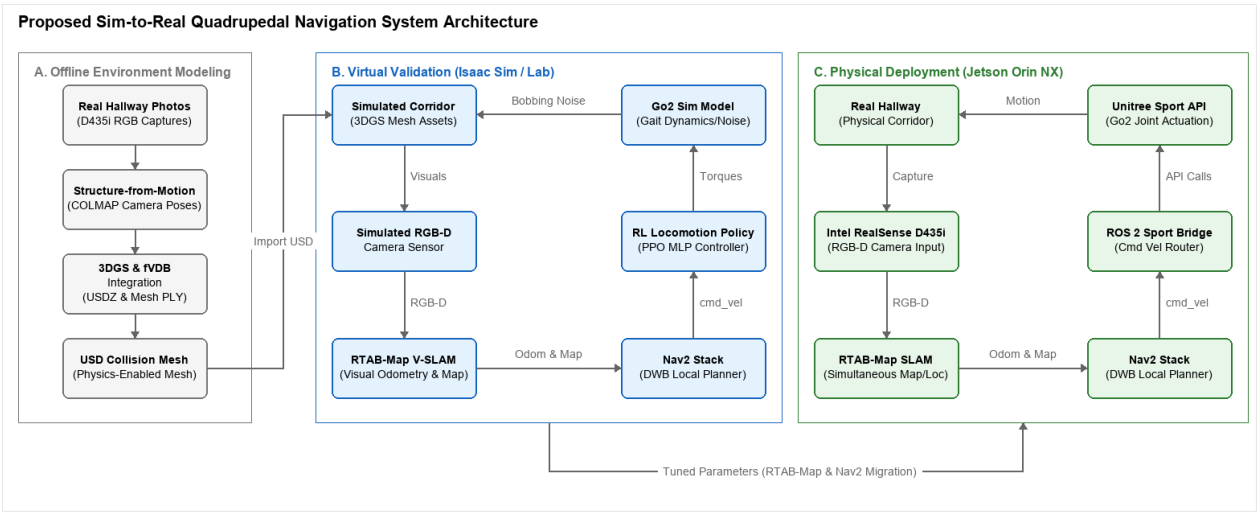


Fig. 1. Overall architecture of the proposed Sim-to-Real quadrupedal navigation framework. (a) In the virtual validation stage, we build a high-fidelity 3DGS environment and train an RL locomotion policy to simulate camera bobbing noise for upper-level navigation tuning. (b) In the physical deployment stage, the RL policy is bypassed, and commands are sent to the Unitree Sport API via a ROS 2 bridge to optimize computational resources on the Jetson Orin NX.

tions in the virtual sandbox.

3. We demonstrate the feasibility of the pipeline by showing that navigation and mapping parameters optimized under simulated visual and dynamic disturbances transfer directly to a physical Unitree Go2 platform without real-world recalibration.

## 2. RELATED WORK

### 2.1 Reinforcement Learning for Quadrupedal Locomotion

Reinforcement learning (RL) has significantly advanced control paradigms for legged systems [1]. Classical model-based methods, such as Model Predictive Control (MPC) and Whole-Body Control (WBC), rely heavily on accurate system dynamics and environmental contact models. Conversely, model-free RL algorithms enable quadrupedal platforms to acquire robust locomotion policies through trial-and-error interactions. The advent of highly parallelized physics simulators, such as NVIDIA Isaac Sim and Isaac Lab, has facilitated the training of deep RL policies across thousands of concurrent environments. This paradigm has enabled successful real-world deployments of agile legged locomotion on systems such as ANYmal and Unitree Go1/Go2. Through domain randomization and curriculum learning, researchers have minimized the dynamics gap, allowing policies to transfer directly from simulation to physical platforms. However, most existing locomotion research focuses primarily on gait stability and disturbance rejection, leaving high-level visual navigation integration and sensor noise characteristics largely unaddressed.

### 2.2 Visual SLAM and Navigation Integration

Mobile robot navigation has classically relied on Light Detection and Ranging (LiDAR) sensors due to their high spatial accuracy and wide field of view. Nonetheless, LiDARs are often cost-prohibitive, heavy, and power-intensive, making them less suitable for payload- and power-constrained quadruped robots. Visual SLAM (VSLAM) utilizing RGB-D cameras offers a lightweight, cost-effective alternative. Among state-of-the-art visual SLAM frameworks, Real-Time Appearance-Based Mapping (RTAB-Map) [2] is widely adopted due to its ability to perform appearance-based loop closure detection combined with pose-graph optimization, producing drift-minimized trajectory estimates. Simultaneously, the ROS 2 Navigation (Nav2) framework [3] has established itself as the industry standard for path planning and obstacle avoidance.

Although recent works have leveraged 3DGS [4] to construct photorealistic digital twins for legged locomotion and navigation, such as VR-Robo [5], they often rely on heterogeneous sensors. For instance, VR-Robo captures environmental scenes using high-end consumer mobile devices (e.g., iPads or iPhones) for offline 3DGS model building, but deploys the trained policy using an action camera (e.g., Insta360 Ace) mounted on the physical quadruped. Deploying digital twins across different hardware setups introduces a device-level perceptual sensor gap, such as mismatched lens distortion models, distinct color profiles, and different field-of-view (FOV) boundaries. In contrast, our pipeline uses the exact same physical RGB-D camera (Intel RealSense D435i) for both the offline 3DGS environment construction and online visual SLAM. This homogeneous sensor setup completely eliminates cross-device perceptual discrepancies and provides a highly integrated, self-contained, low-cost validation pipeline.

## 3. SYSTEM AND LOCOMOTION CONTROL

### 3.1 System Architecture

The proposed framework integrates RTAB-Map Visual SLAM, the Nav2 navigation stack, and a bifurcated locomotion control architecture. The control architecture utilizes a Deep Reinforcement Learning (DRL) locomotion policy for virtual validation and transitions to the robot's native, low-level Sport API for physical deployment. In the simulation phase, the DRL policy—trained via Proximal Policy Optimization (PPO) in Isaac Lab (v2.3.2) on Isaac Sim (v5.1.0)—is employed to generate representative quadrupedal gait dynamics and camera ego-motion (bobbing) noise. High-level velocity commands ($\mathbf{v_{cmd}} = [\mathbf{v_x}, \mathbf{v_y}, \omega_\mathbf{z}]^\mathbf{T}$) computed by the Nav2 local planner are mapped directly to the policy's action space. During real-world deployment, the computationally heavy DRL policy is bypassed to conserve edge computational resources, and Nav2 velocity commands are routed directly to the Unitree Sport API via a custom ROS 2 bridge. This resource-efficient architecture optimizes the computational budget of the Jetson Orin NX. The system architecture is illustrated in Fig. 1.

### 3.2 Locomotion Learning in Isaac Sim

To synthesize representative legged locomotion dynamics and emulate physical camera bobbing noise in simulation, we train an RL policy in NVIDIA Isaac Lab (v2.3.2), built upon the Isaac Sim (v5.1.0) physics engine. The policy functions as a dynamic perturbation generator, producing walking-induced camera oscillations. We utilize the Procedural Terrain Generator in Isaac Lab to construct a training environment with mixed terrains, including uneven ground, ramps, and step obstacles. We employ curriculum learning to dynamically scale the terrain difficulty (e.g., obstacle heights and slope angles) in proportion to the robot's traversal success rate. To enhance the robustness of the policy and bridge the sim-to-real dynamics gap, we apply the following domain randomizations:

- Torso mass randomization: $\Delta m \in [-1.0, +3.0]$ kg.
- Ground friction coefficient: $\mu \in [0.6, 0.8]$.
- External disturbances: impulse force perturbations applied to the robot's center of mass (CoM).

The physics simulation step size is configured to 200 Hz, and the policy control loop executes at 50 Hz (decimation factor of 4).

The policy is trained using the PPO implementation in the PyTorch-based *skrl* library. Both the actor and critic networks are parameterized as Multilayer Perceptrons (MLPs) with hidden layers of $[512, 256, 128]$ and Exponential Linear Unit (ELU) activation functions. The initial learning rate is $1.0 \times 10^{-3}$, regulated by a KL-divergence adaptive scheduler with a target threshold of $0.01$. The discount factor is $\gamma = 0.99$, and the Generalized Advantage Estimator (GAE) parameter is $\lambda = 0.95$.

The reward function is formulated to track velocity references while maintaining stable legged posture. The total reward $R$ is defined as:

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

where:

- $R_{lin} = \exp(-\|\mathbf{v^*_{xy}} - \mathbf{v_{xy}}\|^\mathbf{2}_\mathbf{2}/\sigma^\mathbf{2}_\mathbf{lin})$ is the linear velocity tracking reward, where $\mathbf{v^*_{xy}}$ and $\mathbf{v_{xy}}$ denote the target and actual base linear velocities, respectively ($w_{lin} = 1.5, \sigma_{lin} = 0.25$).
- $R_{ang} = \exp(-(\omega^*_z - \omega_z)^2/\sigma^2_{ang})$ is the angular velocity tracking reward, where $\omega^*_z$ and $\omega_z$ denote the target and actual yaw rates, respectively ($w_{ang} = 0.75, \sigma_{ang} = 0.25$).
- $R_{air}$ denotes the foot clearance air-time reward to promote a distinct, symmetric gait ($w_{air} = 0.25$).
- $P_i$ constitutes the penalty terms enforcing physical constraints: minimizing base vertical linear velocity, roll/pitch oscillations, deviations from nominal joint limits, excessive torque commands, rapid joint accelerations, and unintended structural contact ($w_{p,i}$ are the corresponding penalty weights).

### 3.3 3DGS Environment Construction & Agent Deployment

To construct a high-fidelity virtual verification sandbox, we establish a structured neural reconstruction pipeline that converts physical environments into interactive simulation assets. First, we capture a sequence of high-resolution images of the third-floor corridor of Oseok Hall at Handong Global University. Unlike recent works that capture the environment with high-end smartphone cameras (e.g., iPhones/iPads) possessing aggressive post-processing color grading and autofocus adjustments, we capture all mapping images using the RGB sensor of the same Intel RealSense D435i camera used for online navigation.

These images serve as the input to a Structure-from-Motion (SfM) workflow via COLMAP to resolve sparse camera poses and initial point clouds. Based on these poses, a 3D Gaussian Splatting (3DGS) [4] model is optimized and exported as a USDZ asset for visual rendering, alongside a Polygon Mesh PLY file for coarse geometry. To bridge the gap between photorealistic visualization and physical collision, the reconstructed scene is integrated with NVIDIA's sparse volumetric data framework, *fVDB*, as shown in Fig. 2.

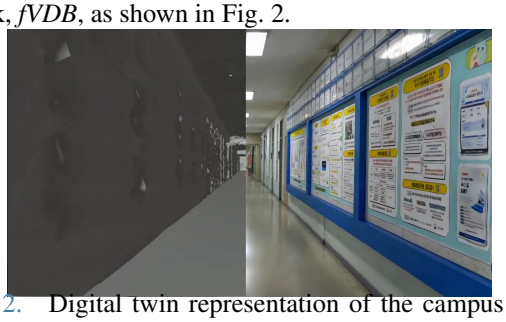

Fig. 2. Digital twin representation of the campus corridor. Left: Sparse volumetric collision mesh generated using fVDB. Right: High-fidelity photorealistic 3DGS render.

We import the combined USDZ and mesh assets into NVIDIA Isaac Sim (v5.1.0). Within the simulator, the NuRec engine utilizes the volumetric *fVDB* representations to compute high-resolution collision boundaries and dynamic depth queries (*GS Collision / Depth*). Finally, a ROS 2 bridge is established to connect the virtual environment to our external navigation stack; this bridge

publishes virtual RGB-D camera sensor streams, visualizes the robot state in RViz, and routes the local planner's velocity commands back to the simulated agent. Because we use the exact same physical D435i sensor for both mapping and navigation, the 3DGS-rendered virtual camera naturally replicates the physical sensor's native lens distortion, white balance, dynamic range, and noise profiles. This eliminates any perceptual domain shift between the virtual validation environment and real-world deployment. The pre-trained DRL locomotion policy is then deployed directly on the Go2 model inside the reconstructed environment without further tuning. Despite the intricate geometric features and narrow corridor boundaries, the policy maintains robust gait characteristics, validating the effectiveness of the domain-randomized training regimen.

## 4. AUTONOMOUS NAVIGATION

### 4.1 Navigation System Configuration

The proposed navigation system consists of a cost-effective, LiDAR-free visual navigation pipeline incorporating an Intel RealSense D435i RGB-D sensor, the RTAB-Map visual SLAM framework [2], and the ROS 2 Nav2 stack. The D435i is rigidly mounted to the front torso of the Unitree Go2. Within the virtual sandbox, a simulated RGB-D sensor replicates matching color and depth profiles. Using a single RGB-D sensor for the entire pipeline also guarantees depth-texture consistency. While frameworks relying purely on monocular RGB reconstruction (e.g., VR-Robo [5]) face geometric holes in textureless regions (such as solid walls or floors) due to photometric ambiguities, our use of the D435i's active depth sensor allows us to project robust metric local costmaps.

The navigation workflow is driven entirely by the synchronized streams. Initially, RTAB-Map processes the RGB stream to extract visual keypoint features (e.g., GFTT/ORB) for frame-to-frame Visual Odometry (VO), loop-closure detection, and pose-graph optimization. The aligned depth stream is then projected into 3D coordinates using these feature coordinates, constructing a local 3D octomap that is subsequently projected onto a 2D occupancy grid map for global path planning. Concurrently, to achieve low-latency local obstacle avoidance without a costly LiDAR instrument, raw depth frames are converted into a pseudo-2D laser scan using the *depthimage_to_laserscan* ROS 2 package. This synthesized scan is published as a */scan* topic to populate the Nav2 local costmap. The Nav2 global planner generates optimal paths on the occupancy grid, while the DWB (Dynamic Window Approach B) local planner utilizes the local costmap to compute steering commands $\mathbf{v_{cmd}}$ that safely route the robot around obstacles. Finally, these commands are transmitted to the active locomotion controller.

A key technical challenge in establishing the simulation-based validation sandbox is resolving the coordinate transform (TF) tree discrepancy between the physics simulator and the standard ROS 2 navigation stack. Conventionally, Nav2 expects a transform chain of *map* $\rightarrow$ *odom* $\rightarrow$ *base_link* (or *base*). In contrast, Isaac Sim defines and tracks the robot's pose relative to the simulation world origin (*world* frame). To satisfy both environments simultaneously and prevent coordinate conflicts or loops, we implement a custom static and dynamic transform bridge. The resulting TF tree is structured as *map* $\rightarrow$ *odom* $\rightarrow$ *world* $\rightarrow$ *base*. Here, RTAB-Map publishes the drift-corrected *map* $\rightarrow$ *odom* transform, while Isaac Sim's OmniGraph publisher links the odometry to the simulator's global frame, and the robot's state publisher completes the transform to the robot's physical base. This unique transform chain enables seamless compatibility, allowing standard ROS 2 navigation algorithms to operate within the virtual sandbox without code modification.

### 4.2 Validation in 3DGS Environment

Prior to real-world deployment, the integrated navigation stack was validated in the 3DGS-based virtual corridor, which replicates the actual corridor dimensions of approximately 80 m in length and 2.5 m in width.

During simulation, the policy network receives proprioceptive and exteroceptive observations at each time step. Proprioception comprises joint positions and velocities, base angular velocity, and gravity vector projection. Exteroception is emulated using a virtual height-scan grid (1.6 m $\times$ 1.0 m at a resolution of 0.1 m) centered on the robot base. Ray-casting queries the reconstructed 3DGS collision mesh to feed local elevation profiles to the policy, enabling adaptive foot placement. The reference command vector $[v_x, v_y, \omega_z]^T$ is supplied directly by the DWB local planner, which coordinates collision-free paths within the virtual environment as visualized in Fig. 3.

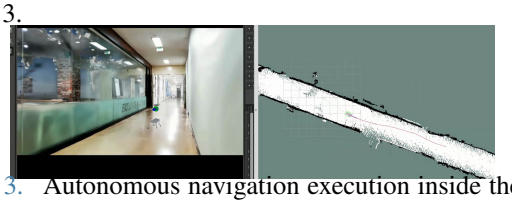

Fig. 3. Autonomous navigation execution inside the reconstructed virtual environment, showing the simulated quadruped robot in Isaac Sim (left) and path planning/costmap synchronization in RViz (right).

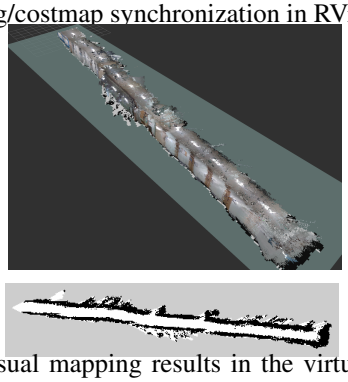

Fig. 4. Visual mapping results in the virtual sandbox. Top: Reconstructed 3D point cloud map from RTAB-Map. Bottom: Projected 2D occupancy grid map.

To evaluate obstacle avoidance, virtual box obstacles were strategically placed along the corridor. As shown in Fig. 3, the local planner dynamically recalculated trajectories around the obstacles, and the RL locomotion pol-

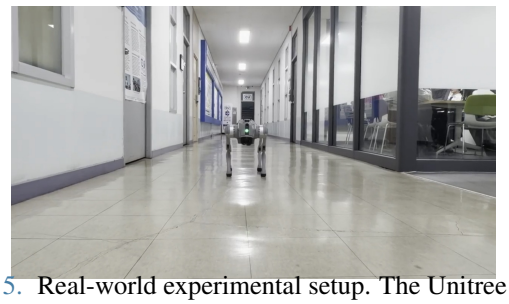

Fig. 5. Real-world experimental setup. The Unitree Go2 robot is equipped with an Intel RealSense D435i and Jetson Orin NX, running the entire navigation stack on-board.

icy adjusted its walking pattern to negotiate the detours without collision. The resulting 3D point cloud map and projected 2D occupancy grid map are shown in Fig. 4.

### 4.3 Physical Deployment and Results

Following virtual verification, the visual navigation pipeline was deployed on a physical Unitree Go2 Edu Plus quadrupedal platform in the actual third-floor corridor of Oseok Hall at Handong Global University. The onboard computing hardware consists of an NVIDIA Jetson Orin NX (operating in 20W power mode, equipped with 16GB RAM) and the robot's built-in expansion dock, both running Ubuntu 20.04 and ROS 2 Foxy. To satisfy strict computational resource constraints, the PyTorch-based DRL locomotion policy was bypassed during physical deployment. Instead, the local trajectory commands ($\mathbf{v_{cmd}}$) generated by Nav2 were mapped directly to the robot's proprietary low-level motor controller via the Unitree Sport API. A lightweight ROS 2 bridge was developed on the Jetson Orin NX to route these commands, thereby reserving processing resources exclusively for RTAB-Map visual SLAM and Nav2 planning. The Intel RealSense D435i depth camera was directly connected to the Jetson Orin NX via USB 3.0.

A major practical challenge in this physical deployment is the operating system (OS) and ROS 2 version mismatch between the virtual simulation environment and the physical robot setup. The simulation validation environment was developed on Ubuntu 24.04 and ROS 2 Jazzy, whereas the physical robot's onboard computing environment is restricted to Ubuntu 20.04 and ROS 2 Foxy due to manufacturer firmware dependencies. To bridge this disparity, the visual navigation stack (RTAB-Map and Nav2) was migrated from ROS 2 Jazzy to ROS 2 Foxy to run natively on the physical Jetson Orin NX. Despite this cross-version migration, the overall visual navigation flow and architecture remain identical to those tested in the simulation environment. This compatibility allowed the navigation parameters optimized under the simulated RL ego-motion noise (specifically, the costmap inflation radius, voxel grid resolution, and visual loop-closure matching thresholds) to be directly applied to the physical Foxy platform, achieving a successful direct Sim-to-Real transfer without parameter recalibration.

As shown in Fig. 5, RTAB-Map successfully executed loop closure detection in the physical environment, compensating for the high-frequency camera oscillations. Nav2 maintained stable global paths, allowing the robot to traverse the corridor at a target velocity of $0.4$ m/s. The physical results verified that key visual SLAM and costmap parameters optimized under the simulated RL ego-motion noise (e.g., costmap inflation radius, voxel grid resolution, and visual loop-closure matching thresholds) were highly transferable to the physical platform, demonstrating the utility of our simulated testing environment.

## 5. CONCLUSION

This paper presented a practical, dual-stage visual navigation framework for quadrupedal robots. By leveraging a high-fidelity 3D Gaussian Splatting digital twin and simulating quadrupedal locomotion noise via an RL locomotion policy, we established a safe, photorealistic virtual validation sandbox for visual SLAM and path planning. For physical deployment, we bypassed the RL policy and routed commands directly to the robot's built-in Sport API via a custom ROS 2 bridge, optimizing edge computation on the Jetson Orin NX. Real-world experiments on the Unitree Go2 demonstrated that the parameters pre-tuned under simulated bobbing noise transferred successfully without manual in-situ recalibration, achieving robust visual tracking and obstacle avoidance.

Future work will focus on incorporating dynamic obstacle avoidance strategies and optimizing the processing pipeline to support higher-resolution visual feedback on the edge computer.

## ACKNOWLEDGEMENT

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-24683458).

## REFERENCES

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[2] M. Labbé and F. Pomerleau, "Online global loop closure detection for large-scale multi-session graph-based SLAM," *Autonomous Robots*, vol. 34, no. 3, pp. 183–200, 2013.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.

[5] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, "VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion," *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 7875–7882, August 2025.